COMPUTER ORGANIZATION AND ASSEMBLY LANGUAGE

Semester Project Report
BS(CS)-3B

Team Members
Aqsa Riaz (01-134231-100)
Syeda Maleeha Bano (01-134231-092)
Laiba Tauseef (01-134231-033)

SUBMITTED TO:
Ma'am Asma Basit

Department of Computer Science

BAHRIA UNIVERSITY, ISLAMABAD

TABLE OF CONTENTS

MATRIX OPERATION

1. Introduction:

There are many assemblers that are used to convert the assembly language code into machine code that a computer's CPU can execute. Here are some examples of assemblers:

- MASM (Microsoft Macro Assembler)
- TASM (Turbo Assembler)
- NASM (Netwide Assembler)
- GAS (GNU Assembler)

But in this project we have used **MASM Assembler.**

MASM, or Microsoft Macro Assembler, is a powerful assembly language programming tool developed by Microsoft for the x86 microprocessor architecture. It is designed to assist developers in writing efficient and optimized low-level code by providing a variety of macros, directives, and facilities for structuring assembly language programs.

This project involves the development of a 32-bit assembly language program designed to perform fundamental matrix operations using the Irvine32 library. The program is capable of handling addition, subtraction, and multiplication of matrices. It reads two matrices from the user, validates their dimensions, performs the chosen operation, and displays the result. This project serves as an in-depth exploration of using assembly language for practical mathematical computations and emphasizes the importance of understanding low-level programming concepts.

2. Objectives:

The primary objectives of this project are to:

- Develop a method to read and store the dimensions and elements of two matrices from the user input.
- Implement functionality to display the contents of the matrices in a readable, formatted manner.
- Create procedures to perform matrix addition, subtraction and multiplication, ensuring proper handling and validation of input data.
- Design a user interface that allows for the selection of matrix operations, through a menu system.

3. Functionality:

1. Reading Matrix Dimensions and Data:

- The user is prompted to enter the number of rows and columns for the matrix, followed by the elements of the matrix.
- The same process is repeated for the second matrix.

2. Displaying matrices:

- After reading each matrix, program displays it in a formatted manner, showing the user the entered data.

3. Menu and Choice Handling:

- A menu is displayed with options for matrix operations: addition, subtraction, multiplication and exit.
- The user selects an operation by entering the corresponding choice number.

4. Matrix operations:

- **Matrix Addition:**
  - If the user chooses addition, the program checks if the matrices have the same dimensions.
  - If the dimensions match, the program adds corresponding elements and displays the result.
  - If the dimensions do not match, an error message is displayed.

- **Matrix subtraction:**
  - If the user chooses subtraction, the program follows a similar process as addition, subtracting corresponding elements if the dimension match.

- **Matric multiplication:**
  - If the user chooses multiplication, the program checks if the number of columns in the first matrix matches the number of rows in the second matrix.
  - If the dimensions are compatible, the program multiplies the matrices and displays the result.
  - If the dimensions are not compatible, and error message is displayed.

4. Code Structure:

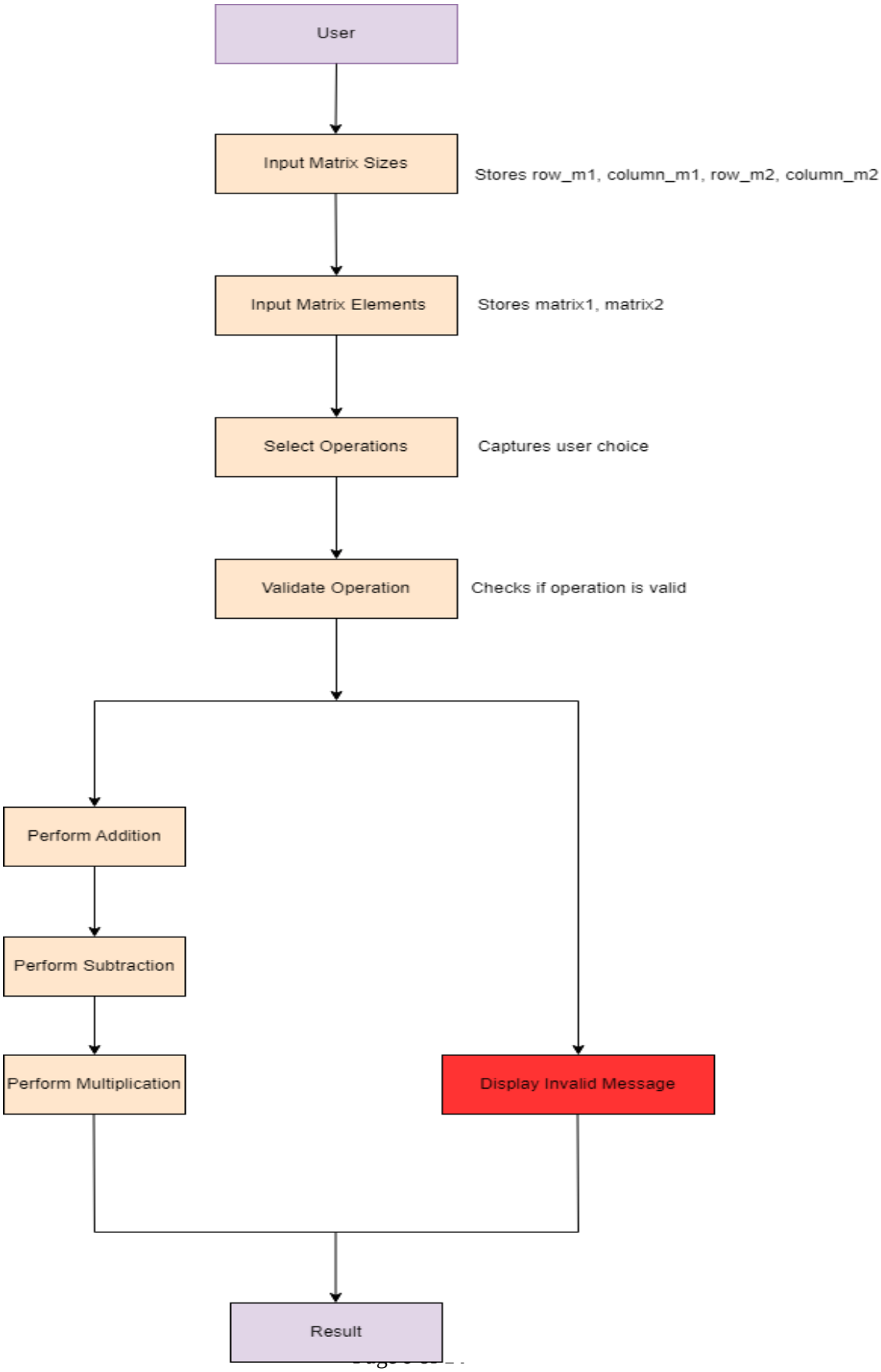
- Data Structure:
  - Defines variables for matrix dimensions, indices and storage.
  - Contains strings for user prompts and message.

- Code Segment:
  - Contains the main procedure (‘main proc’) which drives the overall

program flow.

- Includes separate procedures for each matrix operations (‘matrixAddition proc’, ‘matrixAddition proc’, ‘matrixAddition proc’).

5. Data Flow Diagram:


User
Input Matrix Sizes
Stores row_m1, column_m1, row_m2, column_m2
Input Matrix Elements
Stores matrix1, matrix2
Select Operations
Captures user choice
Validate Operation
Checks if operation is valid
Perform Addition
Perform Subtraction
Perform Multiplication
Display Invalid Message
Result

6. Code:

Addition:

```
matrixAddition proc
    mov eax, row_m1
    mov count, eax            ; Set count to the number of rows in matrix 1
    mov rowindex, 0           ; Initialize row index
    mov columnindex, 0        ; Initialize column index

additionmatrix:
    mov esi, offset matrix1   ; Set ESI to the start of matrix 1
    mov edi, offset matrix2   ; Set EDI to the start of matrix 2
    mov sum, 0                ; Initialize sum to 0

l12:
    mov eax, rowsize_m1
    mov ebx, rowindex
    mul ebx                   ; Calculate offset to the current row
    add esi, eax              ; Add offset to ESI
    mov eax, 0
    mov eax, columnindex
    mov ebx, 4
    mul ebx                   ; Calculate offset to the current column
    mov ebx, eax
```

```
mov eax,0
mov eax,[esi+ebx]
mov sum,eax

mov eax,0

mov eax,rowsize_m2
mov ebx,rowindex
mul ebx
add edi,eax
mov eax,0
mov eax,columnindex
mov ebx,4
mul ebx
mov ebx,eax
mov eax,0
mov eax,[edi+ebx]
mov ebx,sum
add eax,ebx
call writeint
mov eax,0

mov eax,column_m1
cmp columnindex,eax
jb increment_col
jae increment_row

increment_col:

add columnindex,1
mov sum,0
mov edx,offset mystr10
call writestring
jmp l12

increment_row:
call crlf
add rowindex,1
mov columnindex,0
sub count,1
cmp count,0
jbe exit_proc

jmp additionmatrix
```

```
exit_proc:
mov rowindex,0
mov columnindex,0
ret
matrixAddition endp
```

Subtraction:

```
matrixSubtraction proc
    mov eax, row_m1
    mov count, eax            ; Set count to the number of rows in matrix 1
    mov rowindex, 0           ; Initialize row index
    mov columnindex, 0        ; Initialize column index

subtractionmatrix:
    mov esi, offset matrix1   ; Set ESI to the start of matrix 1
    mov edi, offset matrix2   ; Set EDI to the start of matrix 2
    mov x, 0                  ; Initialize x to 0

l13:
    mov eax, rowsize_m1
    mov ebx, rowindex
    mul ebx                   ; Calculate offset to the current row
    add esi, eax              ; Add offset to ESI
    mov eax, 0
    mov eax, columnindex
    mov ebx, 4
    mul ebx                   ; Calculate offset to the current column
    mov ebx, eax
    mov eax, 0
    mov eax, [esi+ebx]        ; Load element from matrix 1
    mov x, eax                ; Store element in x

    mov eax, 0

    mov eax, rowsize_m2
    mov ebx, rowindex
    mul ebx                   ; Calculate offset to the current row in matrix 2
    add edi, eax              ; Add offset to EDI
    mov eax, 0
    mov eax, columnindex
    mov ebx, 4
    mul ebx                   ; Calculate offset to the current column in matrix
2
    mov ebx, eax
```

```
mov eax,0
mov eax,[edi+ebx]
mov ebx,x
sub ebx,eax
mov eax,ebx
call writeint
mov eax,0
mov ebx,0
mov eax,column_m1
cmp columnindex,eax
jb increment_col1
jae increment_row1

increment_col1:

add columnindex,1
mov x,0
mov edx,offset mystr10
call writestring
jmp l13

increment_row1:
call crlf
add rowindex,1
mov columnindex,0
sub count,1
cmp count,0
jbe exit_proc1
jmp subtractionmatrix

exit_proc1:
mov rowindex,0
mov columnindex,0
ret
matrixSubtraction endp
```

Multiplications:

```
matrixMultiplication proc
    mov sum, 0                ; Initialize sum to 0
    mov rowindex, 0           ; Initialize row index
    mov columnindex, 0        ; Initialize column index
    mov r2, 0                 ; Initialize row index for matrix 2
    mov c2, 0                 ; Initialize column index for matrix 2
```

```
multiplicationmatrix:
    ; Initialize sum for each element in the result matrix
    mov sum, 0

compute_element:
    ; Calculate address for matrix1
    mov esi, offset matrix1
    mov eax, rowsize_m1
    mov ebx, rowindex
    mul ebx             ; eax = rowsize_m1 * rowindex
    add esi, eax        ; esi points to the start of the current row in matrix1
    mov eax, r2
    mov ebx, 4
    mul ebx             ; eax = r2 * 4
    add esi, eax        ; esi points to matrix1[rowindex][r2]
    mov eax, [esi]      ; eax = matrix1[rowindex][r2]
    mov product, eax

    ; Calculate address for matrix2
    mov edi, offset matrix2
    mov eax, rowsize_m2
    mov ebx, r2
    mul ebx             ; eax = rowsize_m2 * r2
    add edi, eax        ; edi points to the start of the current row in matrix2
    mov eax, columnindex
    mov ebx, 4
    mul ebx             ; eax = columnindex * 4
    add edi, eax        ; edi points to matrix2[r2][columnindex]
    mov eax, [edi]      ; eax = matrix2[r2][columnindex]
    imul eax, product   ; eax = matrix1[rowindex][r2] * matrix2[r2][columnindex]
    add sum, eax        ; sum += eax

    ; Move to the next element in the row of matrix1
    mov eax, column_m1
    cmp r2, eax
    jl next_element
    jmp output_element

next_element:
    inc r2
    jmp compute_element

output_element:
    ; Output the computed sum
    mov eax, sum
```

```
    call writeint
    mov edx, offset mystr10
    call writestring

    ; Reset r2 for the next computation
    mov r2, 0

    ; Move to the next column of the result matrix
    mov eax, column_m2
    cmp columnindex, eax
    jl next_column

    ; Move to the next row of the result matrix
    call crlf
    inc rowindex
    mov columnindex, 0
    mov eax,row_m1
    cmp rowindex, eax
    jl multiplicationmatrix
    jmp end_multiplication

next_column:
    inc columnindex
    jmp multiplicationmatrix

end_multiplication:
    ret
matrixMultiplication endp
```

7. Output:

```
Rowsize : 2

Columnsize : 3

1
2
3

4
5
6

Matrix 1 :
+1    +2    +3
+4    +5    +6
Rowsize : 2

Columnsize : 3

5
1
2

5
3
4

Matrix 2 :
+5    +1    +2
+5    +3    +4
1.Addition
2.Subtraction
3.Multiplication
4.Exit
Enter your choice 2
Matrix1 - Matrix2 :
-4    +1    +1
-1    +2    +2

C:\Users\haide\source\repos\coallab\Debug\coallab.exe (proc
To automatically close the console when debugging stops, en
```

8. Educational Value:

This project emphasizes the importance of low-level programming concepts by:

- **Memory Management:** Understanding how to allocate, access, and manipulate memory locations directly.
- **Instruction Set Utilization:** Using the assembly language instructions effectively to perform arithmetic operations, control program flow, and manage data.
- **Procedure Calls:** Structuring the program using modular procedures to handle different tasks, such as reading input, performing calculations, and displaying output.

9. Practical Application:

The project showcases the practical use of assembly language in performing complex mathematical operations, which are fundamental in various computational tasks. It demonstrates how assembly language can be used to:

- Directly interact with the hardware.
- Optimize performance for specific tasks.
- Gain a deeper understanding of how high-level operations are executed at the machine level.

10. Future Work:

While the current implementation of the matrix operations program is functional and demonstrates basic matrix manipulations, there are several areas for potential enhancement and expansion. Future work could include the following improvements:

- **Transpose Operation:** Implement a procedure to calculate the transpose of a matrix, which involves swapping rows with columns.
- **Determinant Calculation:** Add functionality to compute the determinant of a square matrix, which is essential in various linear algebra applications.
- **Inverse Matrix Calculation:** Develop a method to find the inverse of a matrix, if it exists, using techniques such as Gaussian elimination.

11. Conclusion:

By developing a program that performs matrix operations in assembly language, this project provides an in-depth exploration of low-level programming. It highlights the intricacies of managing memory and executing arithmetic operations directly on the hardware, reinforcing the importance of assembly language in understanding the fundamentals of computer science. The structured approach of the program also sets a foundation for further expansion and enhancement, allowing for additional operations and optimizations in future projects.

https://github.com/aqsariazshaikh/Matrix-Operation-Project/blob/658e127f132c8793a3df40bcddf395cf6b36004b/IMG_3768%20(1).mp4

THE END